Microsoft Azure

El manual de modernización de aplicaciones

Una guía para modernizar aplicaciones locales y heredadas



Contenido

03

Modernice aplicaciones con confianza

04 Resultados comprobados con modernización basada en agentes

04 Cuatro patrones probados para aplicaciones nativas de la nube seguras y escalables

05

Comience y evalúe su patrimonio digital

05 Exploración acelerada para evaluar la viabilidad

05 Herramientas para una evaluación rápida

06 Modernización con GitHub Copilot

06 Agente de migración de Azure Copilot

06 Utilice un marco rápido de priorización

07

Seleccione la arquitectura de destino correcta

08 Contenedores

09 Centrado en API

10 Controlado por eventos

11 Microservicios

12

Planifique su estrategia de datos

13

Elija un patrón de modernización

14 Rehospedaje

15 Cambiar de plataforma

16 Refactorización

17 Servicios de Azure de un vistazo

18

Modernice las aplicaciones con contenedores: Un marco para la toma de decisiones

20

Ejecute y optimice

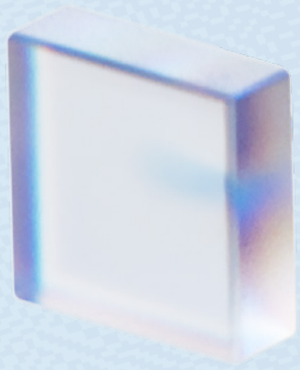
20 Principales prácticas de modernización

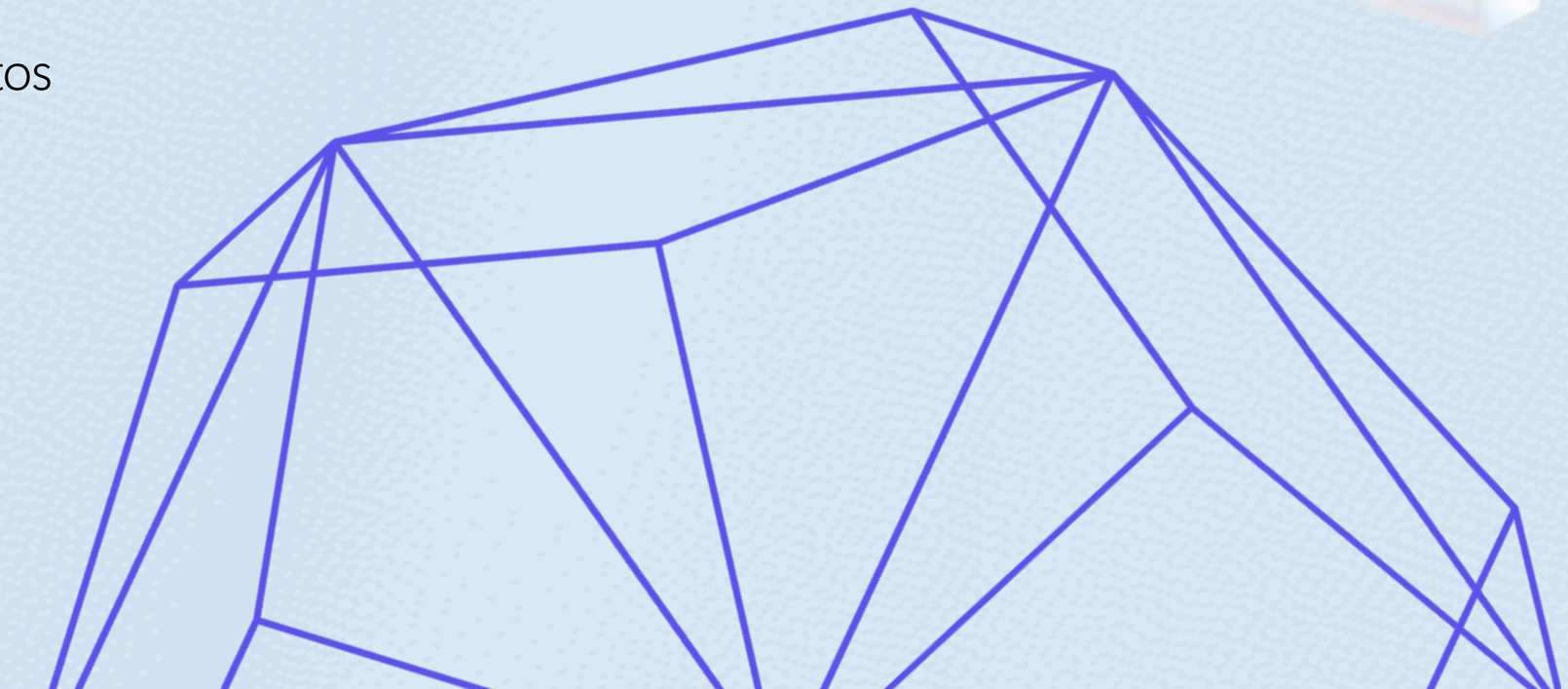
21 Optimización de los costos

21 Impulso de la mejora continua

22

Próximos pasos





Modernice aplicaciones con confianza

Las organizaciones que han modernizado sus patrimonios de aplicaciones están experimentando un 66 % más de rendimiento por cada dólar en cargas de trabajo de procesamiento de transacciones.[1] La preparación para la IA (es decir, la capacidad de integrar servicios inteligentes, ejecutar flujos de trabajo basados en agentes y aprovechar datos en tiempo real) es ahora el principal motor empresarial para la modernización de las aplicaciones.

Durante mucho tiempo, la modernización ha sido un proceso lento, arriesgado y que requiere muchos recursos. Eso está cambiando. Las herramientas de IA con agente ahora pueden evaluar, planificar, actualizar y migrar aplicaciones en cuestión de días en lugar de meses, gestionando transformaciones de código, análisis de dependencias, validación de compilaciones e implementación de Azure dentro del IDE del desarrollador. La modernización ya no es el cuello de botella.

> La pregunta no es si es necesario modernizar. Se trata de qué cargas de trabajo se migran primero y qué capacidad de IA liberan esas migraciones.

El manual de modernización de aplicaciones le permite a pasar de la planificación a tácticas específicas para implementar patrones nativos de la nube. Guías de decisión, listas de comprobación y marcos prácticos le permiten evaluar su portfolio de aplicaciones, seleccionar el destino en la nube adecuado y ejecutar la modernización de aplicaciones con un mínimo de riesgo.

4 veces

más rapidez en la modernización general con la IA con agente[2]

Un 70 %

menos de tiempo de migración con ejecución guiada[3]

Implementación de capacidad de IA más rápida después de la modernización

Resultados comprobados con modernización basada en agentes

Con la IA con agente, la modernización se convierte en una capacidad persistente del sistema en lugar de un programa que sucede una sola vez.

- Hasta un **70 % menos de tiempo de migración** con planificación y ejecución guiadas por IA.[4]
- La actualización de aplicaciones heredadas requiere un **50 % menos de esfuerzo**.[5]
- **Más de 500 000 líneas de código modernizadas en semanas,** no años, en una empresa Fortune 500.[6]
- **Modernización general 4 veces más rápida** con herramientas de IA con agente.[7]

No importa dónde comience, existe una ruta comprobada para avanzar.

Los agentes abarcan .NET (hasta .NET 10), Java (de 8 a 25, de EE a Jakarta), sistema central (COBOL, CICS, JCL), VMware, Windows, Linux y todas las principales bases de datos. La migración consciente de las aplicaciones descubre, agrupa y migra cargas de trabajo dependientes como unidades cohesivas, en lugar de migrar un servidor a la vez.

Cuatro patrones probados para aplicaciones nativas de la nube seguras y escalables

Son cuatro patrones de arquitectura los que se destacan por su valor empresarial y técnico:

- Contenedorización
- Un enfoque centrado en API
- Diseños basados en eventos
- Microservicios

Estos patrones allanan el camino hacia capacidades avanzadas como la integración de la IA, los flujos de trabajo basados en agentes, el análisis en tiempo real y la automatización de DevOps. Cada uno sienta la base para insertar servicios inteligentes, desde la generación aumentada por recuperación (RAG) hasta agentes autónomos que orquestan procesos de negocio.

Puede acelerar la transformación utilizando la **modernización con GitHub Copilot**, que automatiza el análisis de código, las actualizaciones de marcos de trabajo, la contenedorización y la migración a Azure directamente en Visual Studio y Visual Studio Code.



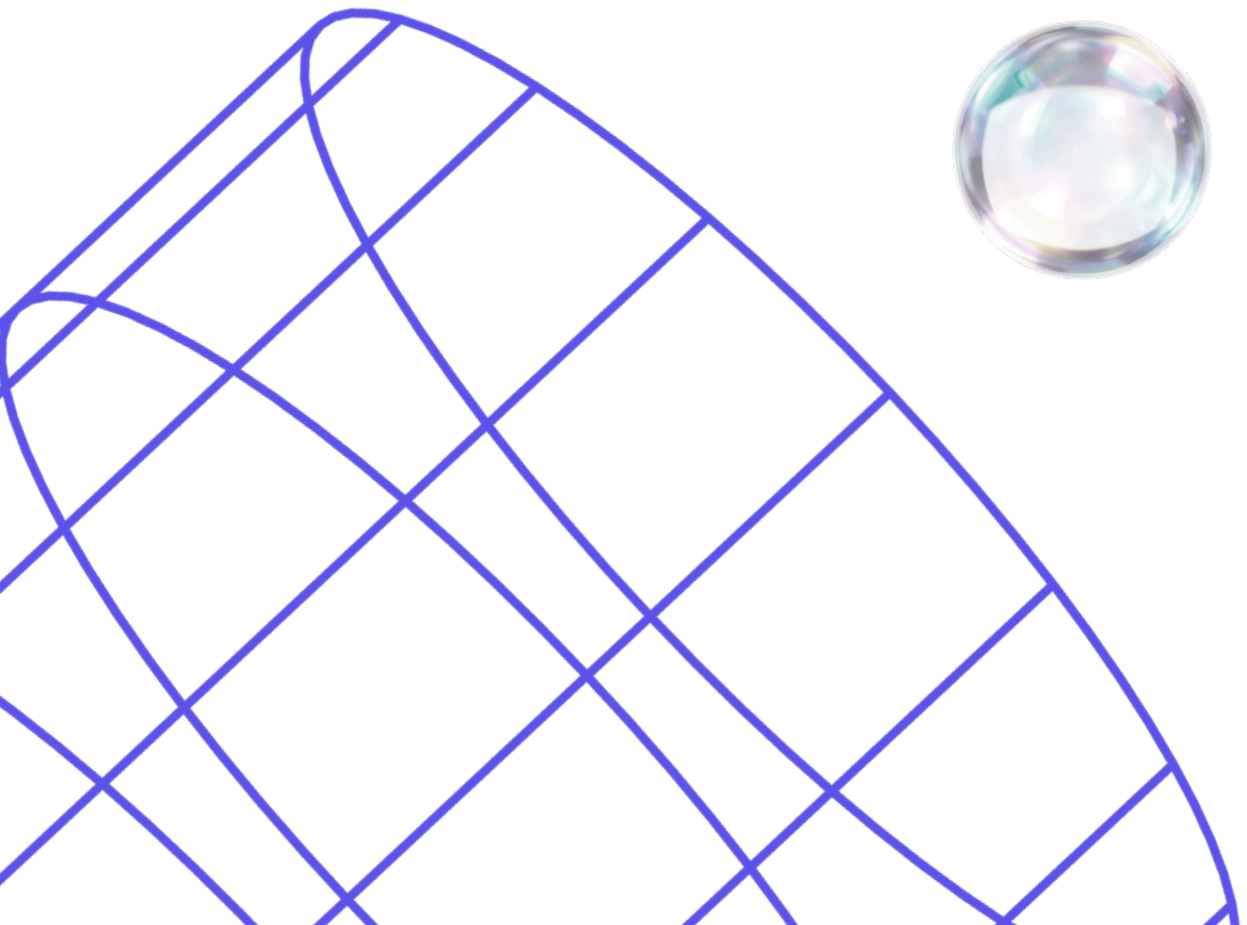
Comience y evalúe su patrimonio digital

Una lista priorizada de aplicaciones muestra dónde la modernización puede generar el mayor valor para su organización. No obstante, muchos equipos se frenan en este paso, porque les hace falta evaluar de manera integral sus aplicaciones y datos.

Una vista clara y práctica de su patrimonio digital le muestra lo que tiene, cómo está conectado y qué cargas de trabajo ofrecen el mayor valor para la transformación nativa de la nube. También puede evaluar la preparación de la organización. Las habilidades, los horarios y los presupuestos afectan sus prioridades.

Exploración acelerada para evaluar la viabilidad

Comience su proceso de modernización con el descubrimiento guiado por la IA. En lugar de inventarios manuales y evaluaciones estáticas, los agentes de IA analizan su patrimonio de manera inteligente, mapeando aplicaciones, dependencias, puntos de integración y patrones de infraestructura en tiempo real. Identifican oportunidades de modernización, modelan escenarios de costos y resaltan riesgos antes de que se lleve a cabo la migración. Con la priorización inteligente de cargas de trabajo y recomendaciones predictivas relacionadas con las zonas de aterrizaje, puede secuenciar las migraciones de manera estratégica, minimizando conflictos de dependencias y retrabajos inesperados durante la ejecución.



Herramientas para una evaluación rápida

Crear un inventario de aplicaciones y dependencias

Utilice mapas de dependencias y descubrimiento con tecnología de IA. El análisis de dependencias sin agente o basado en agentes agrupa los componentes que deben migrarse juntos para evitar dependencias varadas. Obtiene una lista de activos descubiertos, mapas de dependencias, líneas base de rendimiento, estimaciones de costos e indicadores de preparación.

Evaluar costos y viabilidad de alto nivel

Utilice la evaluación guiada por la IA para obtener estimaciones aproximadas de costos y tamaños, además de una evaluación de idoneidad para la nube. Combínelos con la Calculadora de precios de Azure para obtener información específica del servicio.

Evaluar la modernización en el nivel de código

Examine el código fuente y los archivos de configuración con el fin de evaluar la complejidad y proporcionar recomendaciones de migración. Con agentes de modernización con tecnología de IA, los arquitectos y los propietarios de aplicaciones pueden evaluar varias aplicaciones a la vez, analizando el código, las dependencias y la configuración para generar planes de migración alineados con la nube a gran escala. Esas evaluaciones se integran con herramientas de planificación y fluyen directamente a GitHub, donde los desarrolladores pueden integrarlas en su IDE y comenzar a ejecutarlas de inmediato.

Evaluar el riesgo y la idoneidad

Revise el Marco de buena arquitectura de Azure, una evaluación de autoservicio que le permite asignar una puntuación a la confiabilidad, la seguridad, el costo, las operaciones y el rendimiento de una carga de trabajo, y puede exportar los resultados a una lista de tareas pendientes.

Modernización con GitHub Copilot

La modernización con GitHub Copilot es una capacidad impulsada por agentes que le permite modernizar aplicaciones de principio a fin, desde la evaluación y la planificación hasta la transformación del código, las pruebas y la implementación. Funciona directamente en GitHub y en los flujos de trabajo de los desarrolladores.

En el nivel de aplicación individual, puede:

- Evaluar aplicaciones y dependencias.
- Generar planes de modernización específicos para cada aplicación.
- Transformar el código, la configuración y la infraestructura.
- Validar los cambios e implementarlos directamente en Azure.

A lo largo del flujo de trabajo, Copilot aplica orientación basada en IA en cada fase, lo que ayuda a automatizar las tareas repetitivas, reducir el esfuerzo manual y trabajar directamente dentro de los flujos de trabajo del IDE y GitHub, con control total sobre la revisión y la ejecución.

Agente de migración de Azure Copilot

Con el agente de migración, la modernización con GitHub Copilot va más allá de aplicaciones individuales. Coordina la modernización entre los distintos portfolios mientras evalúa cada aplicación de acuerdo con su arquitectura, dependencias y contexto empresarial.

En su portfolio de aplicaciones, puede utilizarlo para:

- Evaluar la preparación de varias aplicaciones de manera simultánea.
- Generar planes de modernización específicos para cada aplicación.
- Automatizar las actualizaciones para aplicaciones Java y .NET.
- Crear y actualizar incidencias de GitHub, solicitudes de incorporación de cambios e informes para cada aplicación.
- Conectar a los propietarios de aplicaciones, arquitectos y desarrolladores a lo largo del proceso de modernización.

Utilice un marco rápido de priorización

Puede priorizar los candidatos para la modernización con un sencillo sistema de puntuación y clasificación basado en los criterios que son más importantes para su organización. La evaluación de la complejidad técnica y el valor empresarial le permite determinar el enfoque correcto para cada aplicación.

Candidato de ganancia rápida	Baja complejidad técnica, alto valor empresarial	**Modernizar primero**
Candidato estratégico	Alta complejidad técnica, alto valor empresarial	**Planificar cuidadosamente**
Candidato para rehospedaje	Alta complejidad técnica, bajo valor empresarial	**Migrar mediante lift-and-shift ahora, modernizar después**
Aplazamiento	Baja complejidad técnica, bajo valor empresarial	**Evaluar para retiro**

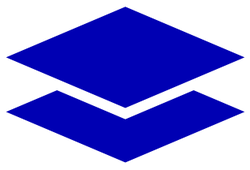
Seleccione la arquitectura de destino correcta

Después de identificar los candidatos para la modernización y asignarles prioridad, el próximo paso es determinar la arquitectura óptima de cómputo. La mayoría de las decisiones se dividen en tres grandes categorías.



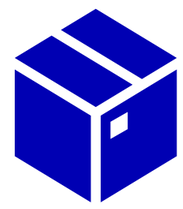
IaaS

Una migración mediante lift-and-shift puede reducir rápidamente los costos del centro de datos y simplificar la migración. Replica en gran medida el entorno existente con un mínimo de cambios en el código. Es una opción adecuada cuando necesita una salida rápida del centro de datos o para el reemplazo al llegar el fin del soporte.



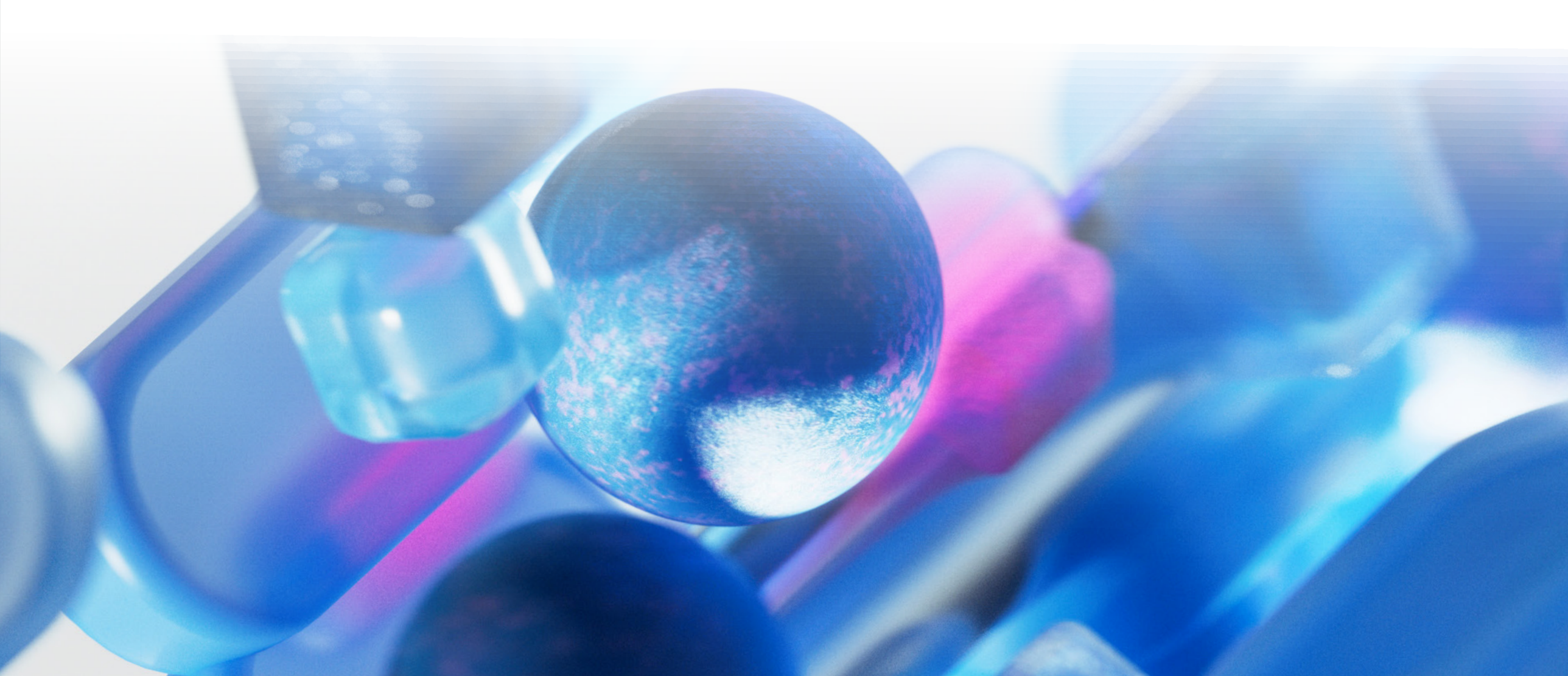
PaaS

Cuando el objetivo es desbloquear capacidades nativas de la nube, puede adoptar servicios de plataforma, lo que rediseña los componentes para mejorar la agilidad y la escalabilidad a largo plazo. Los servicios administrados abstraen la administración de la infraestructura, lo que permite que los equipos se concentren en la innovación y la lógica de aplicaciones.



Contenedores

Para cargas de trabajo que requieren portabilidad, escalabilidad, coherencia y agilidad operativa, los contenedores son un paso natural. La ejecución de contenedores en plataformas administradas combina los beneficios del diseño nativo de la nube con un mínimo de cambios en el código, lo que es ideal cuando se busca rapidez sin tener que reescribirlo todo. Los ecosistemas maduros de .NET, Java y Linux hacen que la contenedorización sea una solución sencilla.



DESTINO → PaaS + contenedores

Contenedores

Los contenedores se han convertido en una manera habitual de empaquetar y ejecutar aplicaciones para realizar la modernización, porque brindan artefactos coherentes y portátiles en todos los entornos y pueden acelerar la adopción de la nube sin que sea necesario reescribir todo. Al empaquetar las aplicaciones y sus dependencias en imágenes contenedorizadas, se obtiene portabilidad y un comportamiento predecible en las distintas plataformas.

Los equipos suelen utilizar herramientas de Docker para crear imágenes y ejecutarlas en entornos de tiempo de ejecución. Además de la portabilidad, los contenedores simplifican prácticas modernas de entrega como las implementaciones inmutables, los lanzamientos declarativos (implementación gradual, azul-verde o de valor controlado) mediante orquestadores, y la integración con canalizaciones de CI/CD. Para aplicaciones web y API que no requieren orquestación de contenedores, Servicio de aplicaciones Azure ofrece un modelo de implementación administrado más sencillo con código o contenedores sin administración de clústeres.

Ejemplos de casos de uso

Aplicación monolítica
Para lograr portabilidad, cambie directamente a una plataforma como servicio administrada (como Servicio de aplicaciones Azure) o contenedorice como primer paso antes de refactorizar.

Servicios web
Utilice contenedores o PaaS administrados por la plataforma para escalar servicios web y hospedarlos en un entorno de ejecución administrado.

Procesos por lotes
Use contenedores para trabajos de corta duración o cargas de trabajo paralelas y ejecútelos en un entorno de ejecución de contenedores administrado para proporcionar aislamiento y portabilidad para tareas programadas o a petición.





DESTINO → PaaS + simplicidad

Centrado en API

El enfoque centrado en API garantiza que los servicios están diseñados para la interoperabilidad desde el principio, lo que reduce el acoplamiento y acelera la innovación entre canales y plataformas. Al dar prioridad a los contratos de API antes de la implementación, puede envolver los sistemas heredados con una fachada de API, unificar los patrones de acceso y unir la comunicación sincrónica y asincrónica. Este enfoque simplifica la integración, admite experiencias omnicanal y prepara su arquitectura para tecnologías emergentes, microservicios y flujos de trabajo basados en eventos.

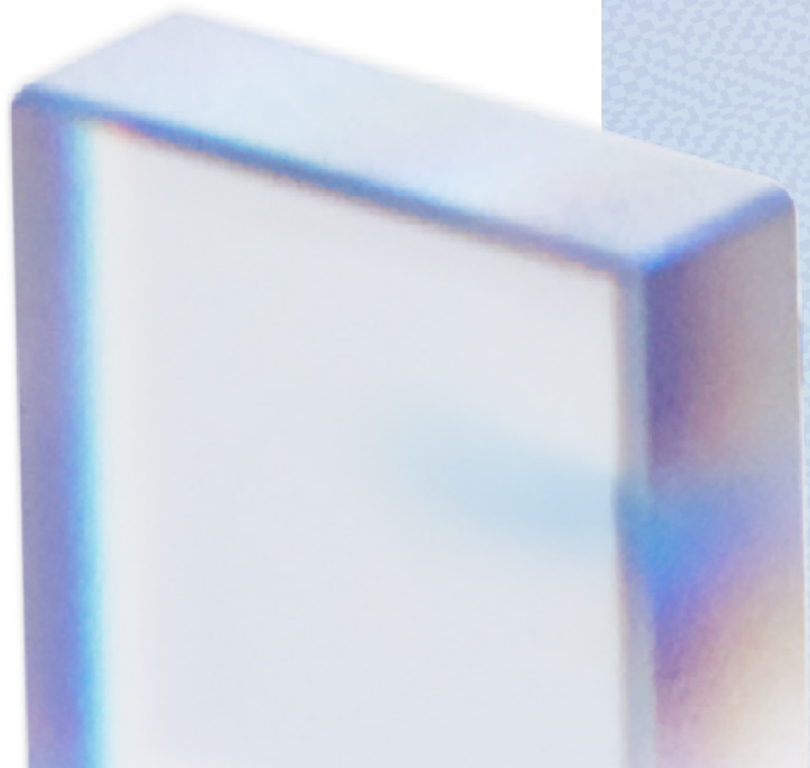
Ejemplos de casos de uso

Aplicación web monolítica
Exponga las funciones empresariales principales como API para admitir aplicaciones móviles e integraciones con socios. Defina API RESTful y envuelva la lógica heredada con una fachada de API hospedada en un servicio de proceso administrado.

Aplicación Java con servicios de SOAP
Reemplace los puntos de conexión de SOAP por API HTTP RESTful o servicios de gRPC, definidos por adelantado.

Aplicación personalizada con procesamiento por lotes
Diseñe API para la ingesta y recuperación de datos antes de refactorizar la lógica de procesamiento por lotes para admitir el intercambio de datos en tiempo real y la integración con otras plataformas.





DESTINO → PaaS + rediseño

Controlado por eventos

Las arquitecturas basadas en eventos promueven la escalabilidad y el acoplamiento flexible al permitir que los servicios reaccionen de manera asincrónica, con lo que son adecuadas para cargas de trabajo de alto rendimiento o en tiempo real. Estas arquitecturas pueden implementarse mediante cómputo sin servidor, donde la plataforma administra el escalado y la ejecución, o combinarse con la contenedorización, donde se controla la configuración del entorno en tiempo de ejecución, las directivas de escalado y los entornos de implementación.

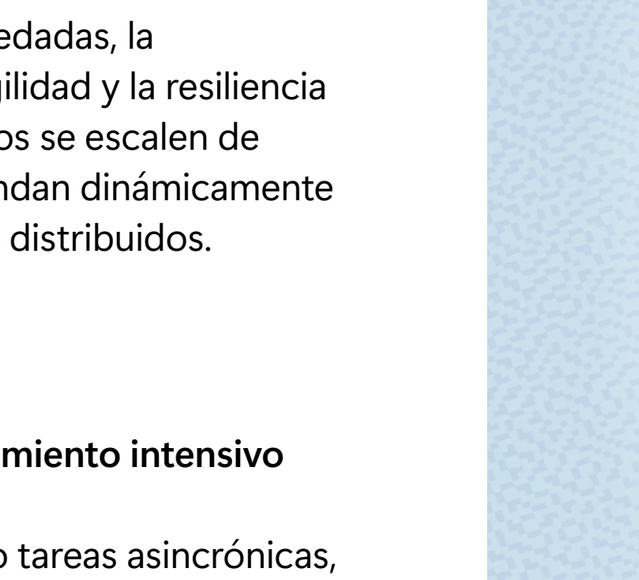
- Si su sistema ya está altamente desacoplado, la arquitectura sin servidor podría ser la opción más simple.
- Al modernizar aplicaciones heredadas, la contenedorización mejora la agilidad y la resiliencia al permitir que los microservicios se escalen de manera independiente y respondan dinámicamente a eventos en distintos entornos distribuidos.

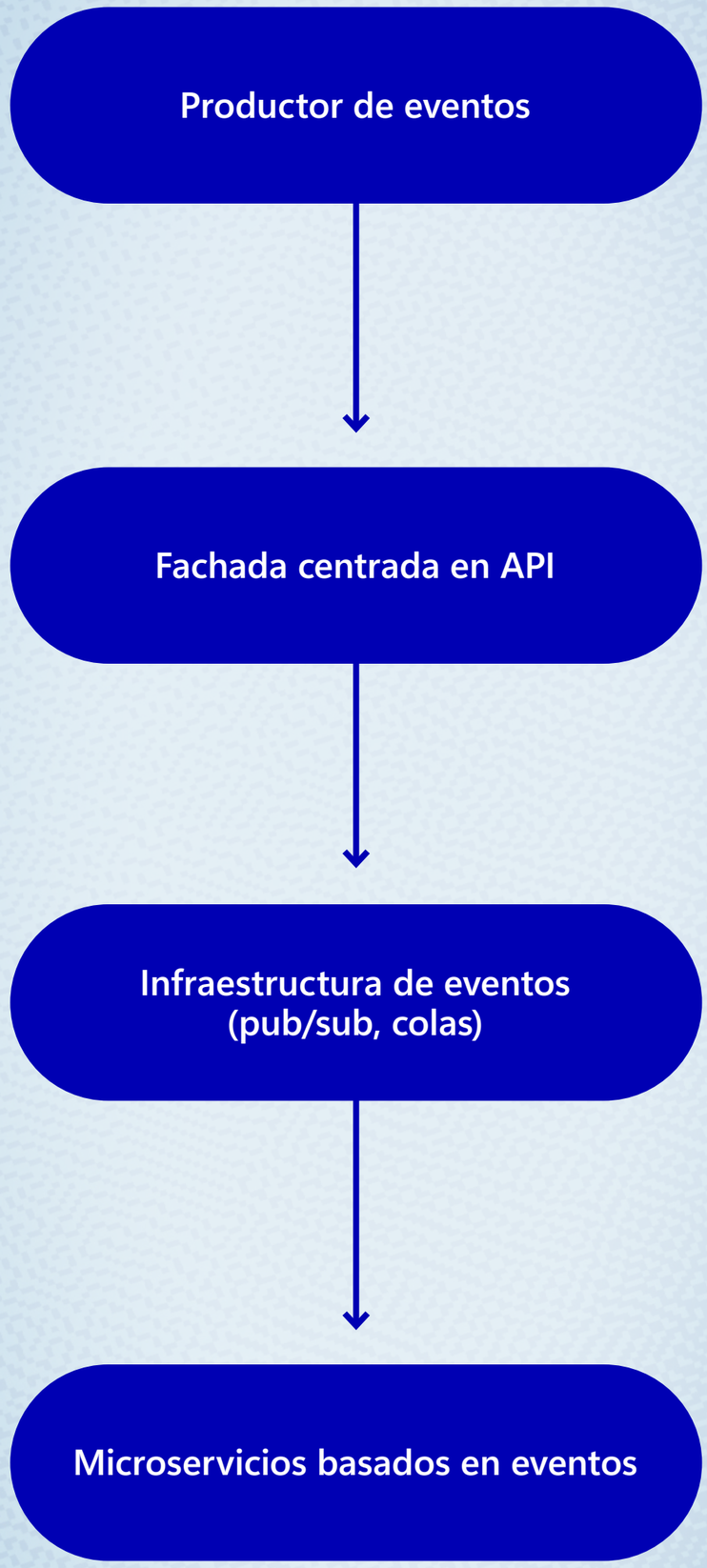
Ejemplos de casos de uso

Aplicación monolítica con procesamiento intensivo de datos
Desacople grandes flujos de datos o tareas asincrónicas, como actualizaciones de inventario minorista, con patrones basados en eventos para lograr escalabilidad.

Sistemas basados en lotes heredados
Modernice los almacenamientos de datos, las canalizaciones ETL o los sistemas de IoT utilizando canalizaciones basadas en eventos para el análisis y la ingesta de streaming.

Flujos de trabajo de IA con agente
Descomponga las tareas con microservicios para emitir eventos y, luego, utilice agentes que reaccionen y orquesten acciones posteriores.





DESTINO → PaaS + rediseño

Microservicios

Como un cambio estratégico que requiere mayor esfuerzo, los microservicios mejoran la confiabilidad y el rendimiento al aislar los errores y permitir el escalado específico, ambos aspectos cruciales para la agilidad empresarial moderna. Al aplicar la descomposición basada en dominios, puede dividir las arquitecturas monolíticas en servicios escalables e implementables de manera independiente. Este enfoque también permite que los equipos elijan los lenguajes de programación, marcos, bases de datos o entornos de tiempo de ejecución más adecuados para cada servicio.

Ejemplos de casos de uso

Plataforma de comercio electrónico
Divida las capas estrechamente acopladas en servicios como Catálogo, Pedidos, Pagos e Inventario. Escale e implemente cada servicio sin que afecte a los demás.

Sistemas críticos
Descomponga una plataforma empresarial central en microservicios; por ejemplo, Administración de pólizas, Reclamaciones, Facturación e Incorporación de clientes.

Flujos de trabajo de IA con agente
Utilice microservicios con un diseño centrado en API para facilitar que los agentes llamen a funciones discretas de manera segura.





Planifique su estrategia de datos

Al seleccionar un patrón de arquitectura, es importante considerar cómo su sistema accederá a los datos y cómo los integrará y protegerá. Cada patrón tiene sus propias ventajas y desventajas en términos de rendimiento, flexibilidad y escalabilidad. Una plataforma de datos unificada garantiza que las aplicaciones, los análisis y la IA trabajen a partir de datos coherentes y de alta calidad, en lugar de silos fragmentados.

La base de datos adecuada para cada patrón depende de los requisitos de rendimiento, coherencia e integración de la carga de trabajo. Las bases de datos administradas en la nube eliminan la sobrecarga operativa de aplicación de revisiones, copias de seguridad y escalado, a la vez que agregan capacidades nativas como vector de búsqueda e integraciones de IA que son esenciales para las aplicaciones inteligentes modernas.

Bases de datos de Azure: entregue aplicaciones y agentes de IA de manera más rápida, con capacidades integradas para IA generativa y cargas de trabajo de LLM

Azure ofrece servicios totalmente gestionados para SQL, NoSQL, PostgreSQL y MySQL, lo que proporciona la base para cargas de trabajo críticas de cualquier tamaño que su organización utiliza para lograr la transformación impulsada por IA y la innovación empresarial.

Con Azure Databases, las empresas obtienen hasta cinco veces más rendimiento, detección inteligente de amenazas y hasta un 58 % menor costo total de propiedad. Las nuevas aplicaciones, agentes de IA y soluciones impulsadas por datos se benefician del desarrollo asistido por GitHub Copilot, recuperación sin contacto, latencia de un solo dígito en milisegundos, capacidades avanzadas de búsqueda vectorial e integraciones con las herramientas de desarrollo, plataformas de análisis y tecnologías de IA más populares.

Más información

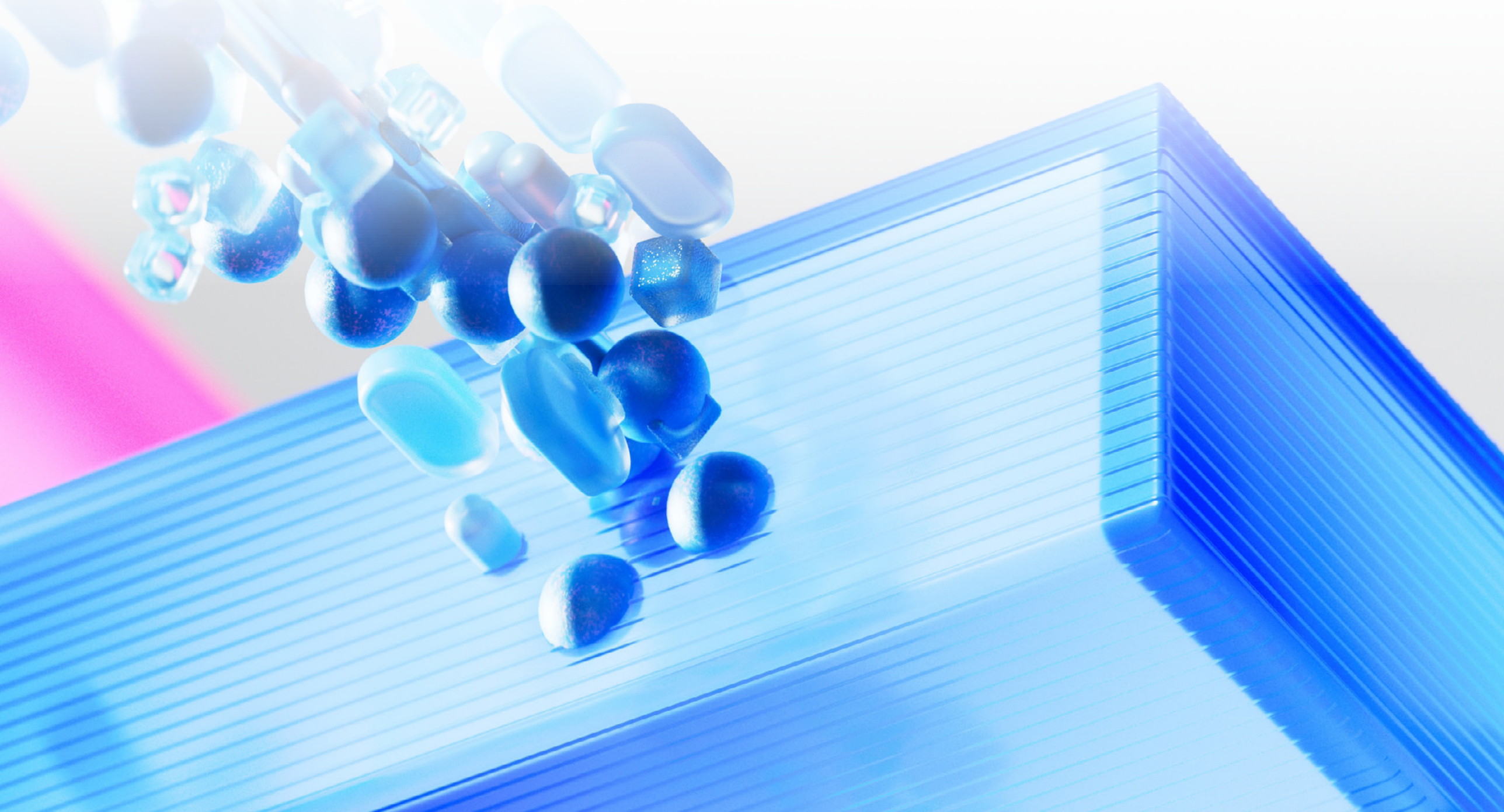


Elija un patrón de modernización

Existen varias versiones de racionalización de la nube (las 6 R en inglés), pero para la modernización de aplicaciones, estos elementos se aplican con mayor frecuencia:

- **Rehospedar** migraciones mediante lift-and-shift a VM.
- **Cambiar de plataforma** migración a servicios administrados (PaaS) sin grandes cambios en el código.
- **Refactorizar** optimizar para los servicios en la nube, modificando el código sin rediseñarlo por completo.
- **Recompilar** reescribir la aplicación desde cero con capacidades nativas de la nube.
- **Retirar** retiro.
- **Retener** dejarlo por ahora tal como está.

Este enfoque ofrece rutas valiosas, pero el rehospedaje, el cambio de plataforma y la refactorización son las estrategias más prácticas y comúnmente adoptadas para modernizar las aplicaciones existentes. Estos enfoques logran un equilibrio entre el esfuerzo y el impacto, lo que los convierte en puntos de partida ideales cuando se desea modernizar de manera incremental y, al mismo tiempo, ofrecer un valor empresarial real.

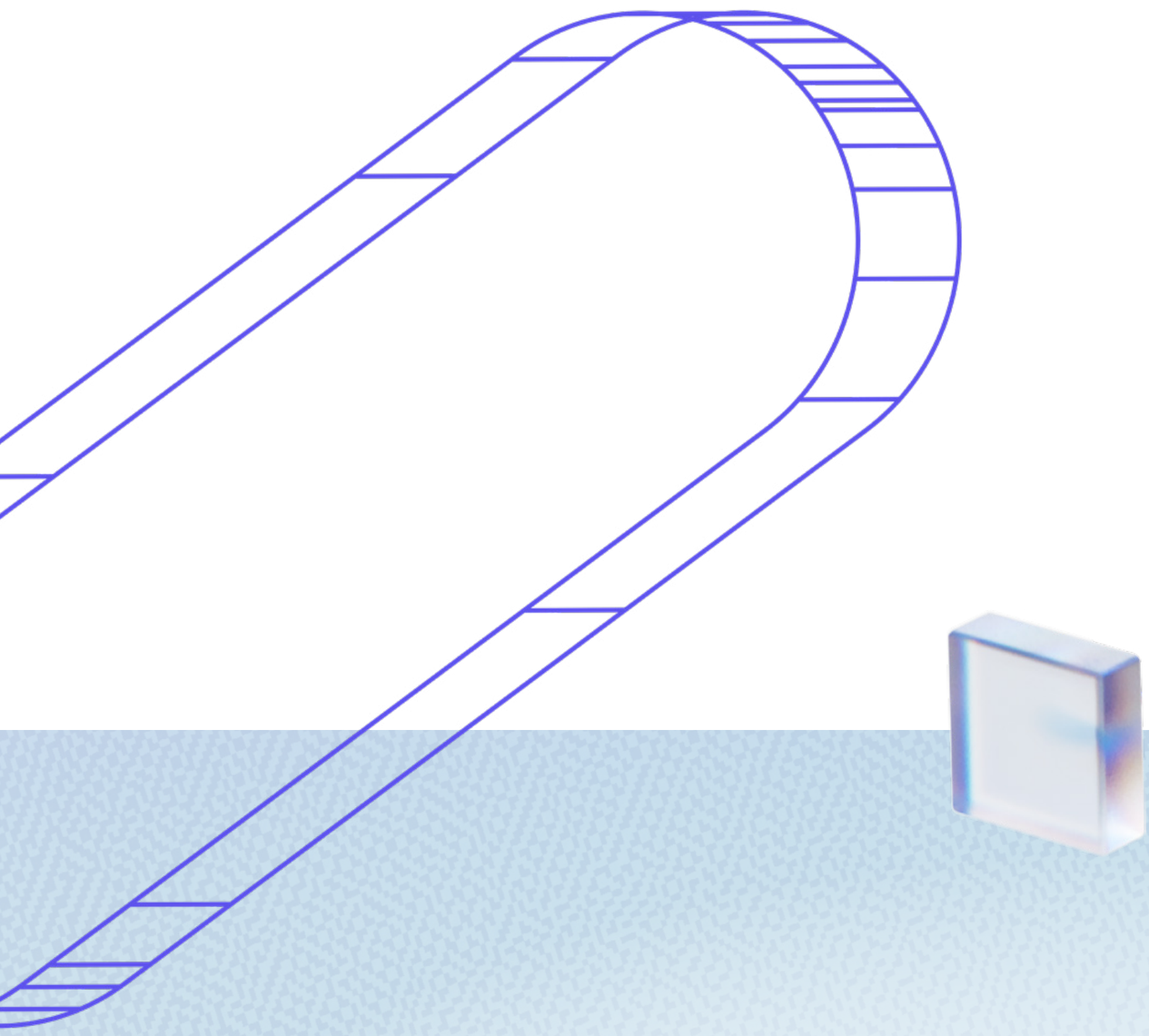


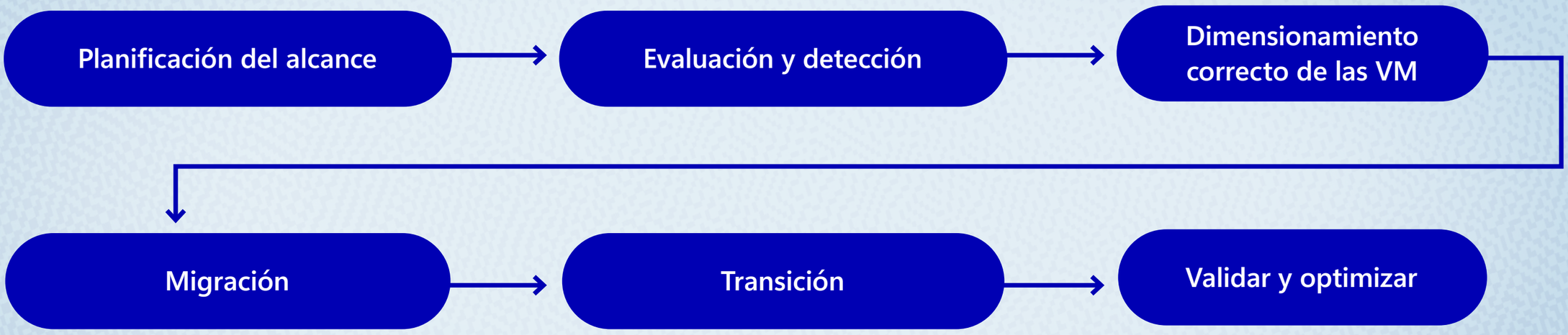
Rehospedaje

El rehospedaje se suele utilizar para comenzar: se trasladan las cargas de trabajo existentes a una infraestructura IaaS equivalente sin modificar la aplicación. Su aplicación todavía no será más moderna, pero el rehospedaje simplifica los pasos posteriores, como el cambio de plataforma, ya que la infraestructura ya está en la nube. La migración suele ser rápida, gracias a herramientas de automatización con tecnología de IA, y ofrece una relación precio-rendimiento significativamente mejor en comparación con la implementación en el entorno local.

Lista rápida de comprobación: Validación después del rehospedaje

- **Funcionalidad de la aplicación:** Confirme que su aplicación funcione según lo previsto después de la migración, incluidas las características principales, los flujos de trabajo y los puntos de integración.
- **Línea base de rendimiento:** Compare el rendimiento posterior a la migración con los valores de referencia previos a la migración para garantizar tiempos de respuesta y rendimiento aceptables.
- **Conectividad:** Compruebe el acceso a la red a servicios dependientes, bases de datos y puntos de conexión externos, incluidos DNS y reglas de firewall.
- **Comprobaciones de seguridad:** Aplique las líneas base de seguridad para VM, habilite Microsoft Defender para la nube y confirme la aplicación de revisiones y el fortalecimiento del SO.
- **Configuración de observabilidad:** Configure Azure Monitor y Log Analytics para supervisar el estado y el rendimiento de las VM y habilite alertas para métricas críticas.





Cambiar de plataforma

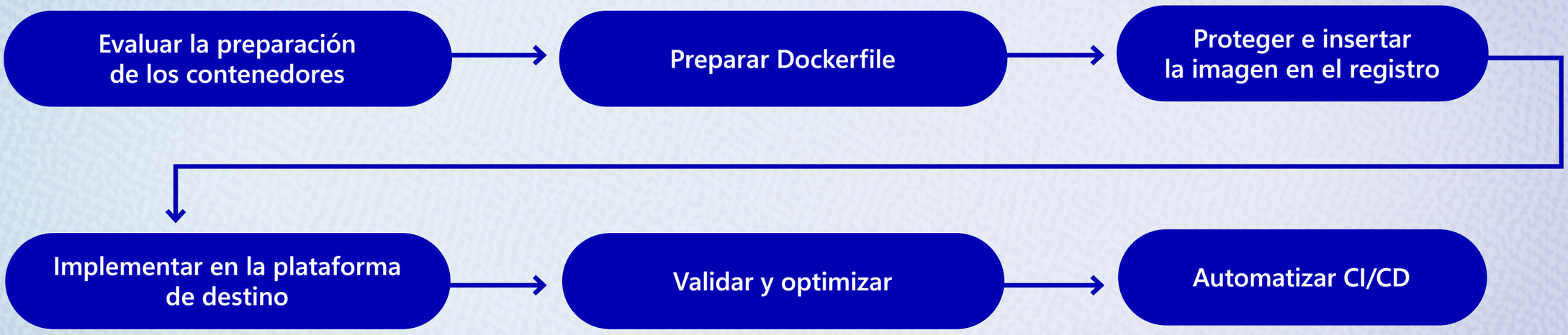
El cambio de plataforma mueve su aplicación a contenedores y la ejecuta en servicios administrados de Azure. Este enfoque transfiere la responsabilidad del aprovisionamiento de la infraestructura, la aplicación de revisiones y el escalado a la plataforma.

Para los equipos, el cambio de plataforma reduce el trabajo de infraestructura manual y respalda prácticas coherentes de DevOps, mientras conserva la mayor parte de la estructura de la aplicación existente. Para la empresa, reduce el costo total de propiedad porque consolida las operaciones y mejora la eficiencia de los recursos, mientras prepara la aplicación para un futuro cambio. También se alinea con los patrones de modernización clave: contenedores para la portabilidad, diseños basados en eventos para la capacidad de respuesta, microservicios para la escalabilidad y diseño centrado en API para la integración.

Lista rápida de comprobación: Validación después del cambio de plataforma

- **Pruebas funcionales:** Asegúrese de que la aplicación funcione correctamente en el entorno administrado de destino (contenedorizado o administrado por la plataforma).
- **Comparación de rendimiento:** Compare el rendimiento contenedorizado con la implementación original para confirmar tiempos de respuesta aceptables y el uso de recursos.
- **Pruebas de integración con la capa de datos:** Compruebe la conectividad y las operaciones con bases de datos, API y otros servicios dependientes después de la contenedorización.
- **Comprobaciones de seguridad:** Valide los controles de seguridad de la plataforma, la postura de vulnerabilidad y el cumplimiento de las directivas de seguridad básicas (por ejemplo, escaneo de imágenes para contenedores o integración con Defender para la nube).
- **Configuración de observabilidad:** Habilite registros, métricas y sondeos de estado para supervisar proactivamente el rendimiento de los contenedores y facilitar la resolución de problemas.

Si la contenedorización forma parte de su estrategia de cambio de plataforma:



De lo contrario, puede cambiar la plataforma de las aplicaciones directamente al Servicio de aplicaciones Azure, lo que simplifica estos pasos a configuración, implementación y configuración de CI/CD, sin necesidad de administración de imágenes y Dockerfiles.

Refactorización

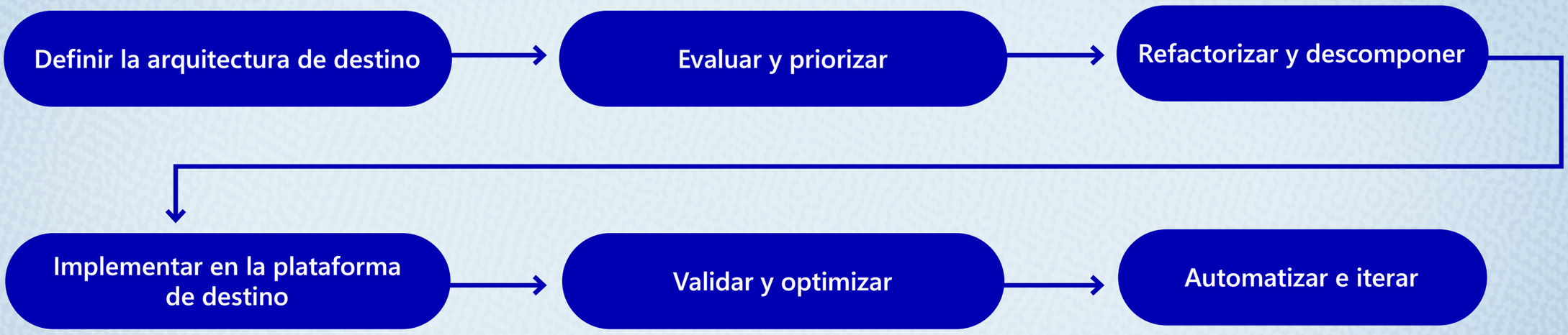
La refactorización actualiza el código de la aplicación para reducir la deuda técnica, mejorar la capacidad de mantenimiento y optimizar el rendimiento. A menudo, implica rediseñar partes para que se ajusten mejor a los patrones de nube y los SDK, por lo que a veces se denomina rediseñar. La refactorización es el camino para salir de la deuda técnica y avanzar hacia la agilidad, ya que permite ejecutar cambios empresariales hasta un 78 % más rápido.[8]

Por ejemplo, separar aplicaciones estrechamente acopladas en componentes modulares permite a los equipos escalar y actualizar partes del sistema de manera independiente. Este enfoque admite diseños centrados en API y basados en dominio, así como microservicios y arquitecturas basadas en eventos.

Una estrategia práctica de descomposición comienza identificando límites claros de servicio, extrayendo primero los componentes de alto valor y manteniendo la compatibilidad con versiones anteriores durante la transición. El lanzamiento incremental contribuye a disminuir el riesgo y brinda valor antes.

Lista rápida de comprobación: Validación después de la refactorización

- **Integridad funcional:** Compruebe que cada servicio refactorizado funcione según lo previsto y que los flujos de trabajo generales de la aplicación sigan siendo coherentes.
- **Referencias de rendimiento:** Compare el rendimiento en el nivel de servicio y de extremo a extremo con las líneas base previas a la refactorización.
- **Pruebas de integración:** Confirme que los nuevos servicios se comunican correctamente con los componentes, las API y los orígenes de datos existentes.
- **Posición de seguridad:** Valide que cada servicio cumpla los requisitos de seguridad, incluida la administración de identidades y de secretos y las comprobaciones de cumplimiento.
- **Preparación para la observabilidad:** Asegúrese de que el registro, las métricas y el seguimiento distribuido estén implementados en todos los servicios.
- **Compatibilidad con versiones anteriores:** Pruebe que los consumidores e interfaces heredados siguen funcionando durante la implementación por fases.



Servicios de Azure de un vistazo

Tanto si decide dar prioridad a la integración centrada en API, la escalabilidad de los microservicios, la portabilidad de los contenedores o plataformas probadas, puede alinear su elección con servicios específicos de Azure y resultados empresariales.

Azure Kubernetes Service (AKS)	Kubernetes totalmente administrado para cargas de trabajo contenedorizadas a gran escala. Ideal para microservicios y necesidades de orquestación complejas.
Servicio de aplicaciones Azure	Plataforma administrada para aplicaciones web y API. La ruta más simple para el cambio de plataforma sin orquestación de contenedores
Azure Container Apps	Contenedores sin servidor para cargas de trabajo de microservicios y basadas en eventos. No se requiere administración de clústeres.
Azure API Management	Gateway de API centralizado para publicar, proteger y supervisar API en distintos entornos.
Azure Service Bus / Azure Event Hubs	Servicios de streaming de eventos y mensajes para arquitecturas desacopladas basadas en eventos.
Modernización con GitHub Copilot	Agente con tecnología de IA para la transformación del código de principio a fin, la planificación de actualizaciones y la implementación de Azure desde el IDE.
Microsoft Defender para la nube	Protección unificada de aplicaciones nativas de la nube en los entornos de desarrollo y tiempo de ejecución.



Modernice las aplicaciones con contenedores: Un marco para la toma de decisiones

Los contenedores son un eficaz habilitador de la modernización, pero el mejor enfoque depende de las características de su carga de trabajo y de sus objetivos empresariales. Si bien la contenedorización suele aportar portabilidad y eficiencia operativa, en algunos escenarios puede ser más conveniente cambiar a otra plataforma o refactorizar primero.

Este marco de decisiones le permite evaluar rápidamente si los contenedores son la opción adecuada para su aplicación y lo guía hacia la ruta de modernización más eficaz.

¿Necesita modernizar aplicaciones críticas de .NET Framework sin reescribirlas?	Considere el uso de una instancia administrada en Servicio de aplicaciones Azure.
¿Es una prioridad la portabilidad entre distintos entornos?	Evalúe si los contenedores son la opción adecuada.
¿Tiene su equipo experiencia en Kubernetes?	Considere AKS para la orquestación completa o Container Apps para una simplicidad sin servidor.
¿Necesita cambios mínimos de código y una migración rápida?	Cambie de plataforma directamente al Servicio de aplicaciones Azure.
¿Se orienta hacia patrones basados en eventos o microservicios?	Opte por contenedores junto con AKS o Container Apps (opción recomendada).

Procedimientos recomendados y medidas de seguridad integradas

Azure incluye medidas de seguridad que facilitan y hacen más segura la implementación de contenedores. Para evitar errores comunes antes de que ocurran:

Asegúrese de usar imágenes base aprobadas que no sean raíz en su Dockerfile.

Aplique Azure Policy para AKS para garantizar que los contenedores no se ejecuten como raíz. Para comprobaciones de seguridad y cumplimiento en tiempo de ejecución, use Microsoft Defender para contenedores.

No codifique las configuraciones de forma rígida.

Externalice la configuración para reducir la exposición y el desfase. Use mecanismos nativos de la plataforma para variables de entorno y almacenes seguros como Azure Key Vault para los valores confidenciales.

Use compilaciones de Docker de varias etapas.

Minimice el tamaño de la imagen, acelere las implementaciones y almacene imágenes optimizadas en Azure Container Registry para el control de versiones y el análisis de vulnerabilidades.

Implemente sondeos de ejecución y de preparación.

Supervise el estado del contenedor y habilite la recuperación automática. Utilice sondeos integrados en AKS o comprobaciones de estado en Servicio de aplicaciones Azure, con Azure Monitor para alertas y visibilidad.

Defina solicitudes y límites de CPU y memoria.

Establezca límites para que los pods no agoten los recursos del nodo. En AKS, estos límites admiten la programación y el escalado predecibles, y Azure Monitor ayuda a hacer un seguimiento del uso.



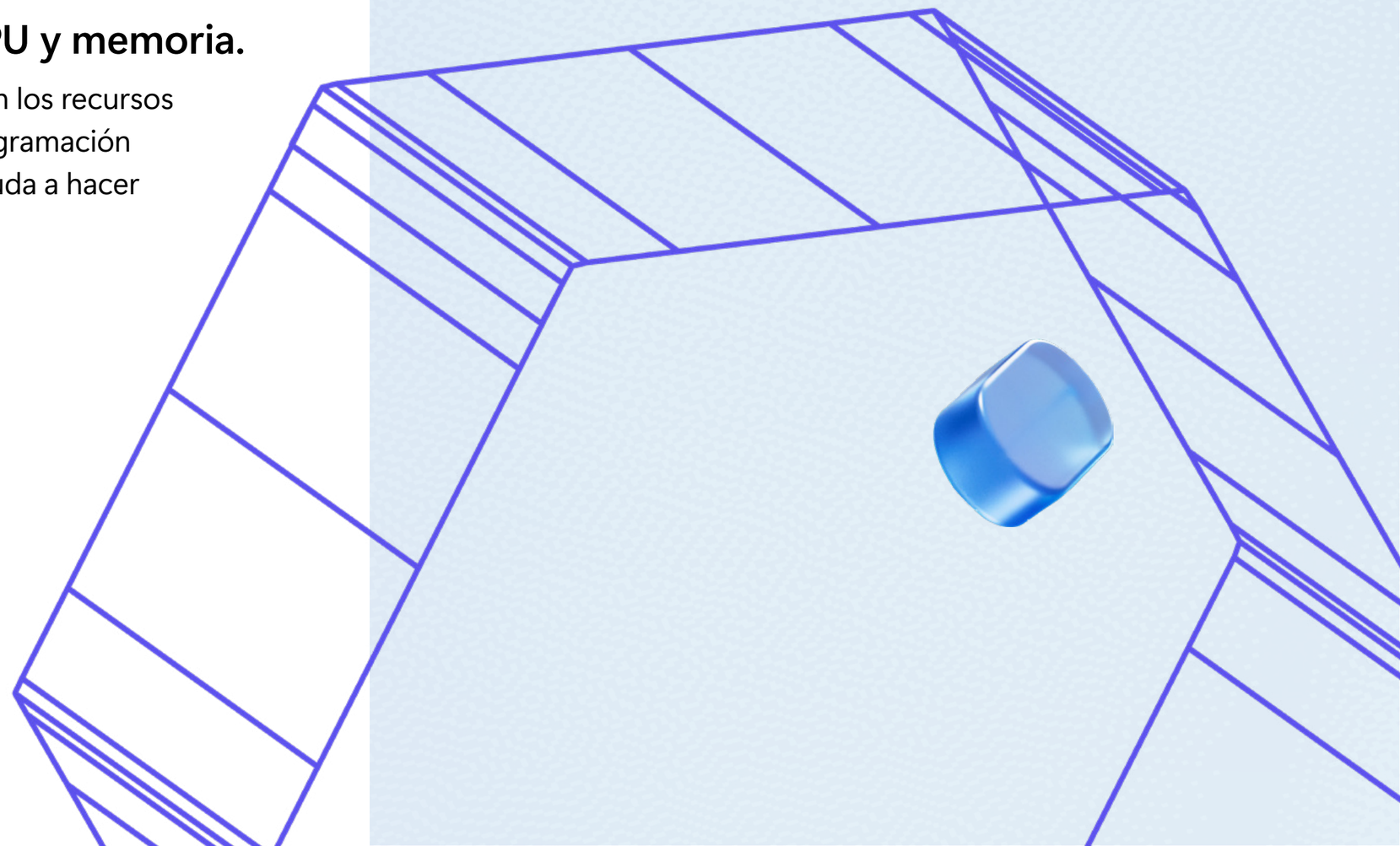
Microsoft y soberanía digital

Microsoft Sovereign Cloud: la oferta más completa de soluciones soberanas en IA, productividad, seguridad y plataforma de nube.

Nuestro enfoque es simple: la soberanía digital es fundamental para nuestra nube y nuestros servicios y se ofrece a través de una combinación bien gobernada de controles técnicos, contractuales y operativos. Dado que cada carga de trabajo y cada organización tiene diferentes niveles de confidencialidad y criticidad, los requisitos de soberanía se adaptan a las necesidades de cada cliente.

Microsoft Sovereign Cloud integra IA, productividad, seguridad y cargas de trabajo en la nube para abarcar entornos públicos, híbridos y desconectados, lo que permite a los clientes aplicar el nivel adecuado de control, capacidad y cumplimiento según la confidencialidad y la criticidad de cada carga de trabajo en la nube, en lugar de depender de un único modelo de implementación.

Más información



Ejecute y optimice

El éxito después de la migración depende de qué lo bien que funcionen, escalen y mantengan su resiliencia sus cargas de trabajo. Sin una optimización adecuada, los costos de la nube pueden dispararse y el rendimiento puede disminuir, lo que socava el valor de la modernización. Una sólida gobernanza, junto con una cultura de mejora continua, constituye la base de una excelencia operativa sostenida.

Principales prácticas de modernización

Algunas prácticas ayudan a proteger sus cargas de trabajo, acelerar la entrega y garantizar la continuidad empresarial en cada ruta de modernización.

- Adopte CI/CD desde el primer día. Las canalizaciones automatizadas reducen el riesgo de implementación y aceleran los ciclos de entrega, lo que permite que los equipos dispongan de más tiempo para la innovación.
- Desplace la seguridad a la izquierda. Integre el análisis de seguridad y las comprobaciones de cumplimiento a la canalización de desarrollo para detectar problemas antes de que lleguen al entorno de producción.
- Establezca estándares de observabilidad. Los registros, las métricas y la trazabilidad distribuida en todos los servicios le brindan la visibilidad que necesita para detectar problemas de manera proactiva y mantener los SLA.
- Gobierne con directiva como código. Automatice los estándares de cumplimiento y configuración para aplicar de manera coherente las protecciones.
- Habilite el lanzamiento incremental. Las estrategias de implementación azul-verde y de valor controlado le permiten lanzar actualizaciones de manera más segura, con capacidad de reversión si surgen problemas.



Optimización de los costos

Existen varias acciones prácticas que puede llevar a cabo y que le permitan reducir los costos de la nube sin que el rendimiento ni la escalabilidad resulten afectados.

- Dimensione correctamente el tamaño de los recursos. Utilice datos de supervisión para identificar VM sobredimensionadas, contenedores y servicios administrados, y ajuste su capacidad para que coincida con la demanda real de carga de trabajo.
- Aplique la capacidad reservada y los beneficios híbridos. Para cargas de trabajo predecibles, los precios reservados y beneficio híbrido de Azure pueden reducir significativamente los costos de cómputo.
- Utilice el escalado automático. Configure directivas de escalado automático para que solo pague la capacidad cuando la necesite, no cuando no la necesite.
- Etiquete y haga un seguimiento del gasto. Implemente un etiquetado coherente de recursos para la asignación de costos y use herramientas de administración de costos para supervisar el gasto por equipo, carga de trabajo o proyecto.

Impulso de la mejora continua

Al medir periódicamente el rendimiento, recopilar información de los equipos y alinear las operaciones con sus objetivos empresariales, puede perfeccionar sus entornos para admitir la escalabilidad nativa de la nube y prepararse para la innovación impulsada por la IA.

La modernización funciona cuando es deliberada. Cuando usa patrones probados, valida cada paso y elige los servicios adecuados para sus cargas de trabajo, puede crear sistemas sostenibles y preparados para el futuro que evolucionen junto a su empresa.



Próximos pasos

La modernización es la base de la transformación para la IA. Los patrones de esta guía proporcionan elementos fundamentales que ayudan a los equipos a avanzar más rápido, confiar en sus resultados y escalar sistemas de manera confiable.

Acelere la adopción de la nube y la IA con soporte experto y financiación

Obtenga orientación integral con Frontier Accelerate

Modernice más rápido con una plataforma moderna lista para la IA

Explore tutoriales en Navegador de habilidades de IA

Diseñe y opere una infraestructura lista para la IA

Siga una ruta guiada y tome el módulo de Microsoft Learn

---

[1] Phenix, Eric. GigaOm Benchmark: Migrating Windows Server Workloads to Azure. GigaOm. 10 de marzo de 2025.
[2] Microsoft's Agentic Tools Reshape Application Modernization for AI Era. LavX News. 13 de febrero de 2026.
[3] Modernizar aplicaciones en días, no meses. Sitio web de GitHub sobre la modernización de aplicaciones.
[4] Acelere la migración y modernización con la IA agéntica. Blog de Azure. 25 de septiembre de 2025.
[5] Modernizar aplicaciones en días, no meses. Sitio web de GitHub sobre la modernización de aplicaciones.
[6] Modernizar aplicaciones en días, no meses. Sitio web de GitHub sobre la modernización de aplicaciones.
[7] Microsoft's Agentic Tools Reshape Application Modernization for AI Era. LavX News. 13 de febrero de 2026.
[8] The Business Value of Migrating and Modernizing IT Estate with Microsoft Azure. IDC. Abril de 2025.

©2026 Microsoft Corporation. Todos los derechos reservados. Este documento se entrega "tal cual". La información y las opiniones expresadas en este documento, incluidas las direcciones URL y otras referencias a sitios web de Internet, están sujetas a cambios sin previo aviso. Usted asume el riesgo de usarlo. Este documento no le otorga derecho legal sobre ninguna propiedad intelectual de ninguno de los productos de Microsoft. Puede copiar y usar este documento para uso interno como material de consulta.